

我是 TOC

GYF

2023 年 12 月

计算理论基础复习提要

李元《计算理论基础》复习提要。

制作者：GYF

GitHub: <https://github.com/23300240002>

第一节课复习资料

一、本节课知识点讲解

1.1 渐近记号 (Asymptotic Notation)

渐近记号用于描述函数在自变量趋于无穷时的增长趋势，是分析算法时间复杂度的核心工具。

大 O 记号 (Big-O) 定义: $f(n) = O(g(n))$ 表示存在正常数 c 和 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \leq c|g(n)|$ 。

含义: $f(n)$ 的增长速度不超过 $g(n)$ 的常数倍, 描述上界。

例子: $3n^2 + 2n + 1 = O(n^2)$ 。

Ω 记号 (Omega) 定义: $f(n) = \Omega(g(n))$ 表示存在正常数 c 和 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \geq c|g(n)|$ 。

含义: 描述下界。

Θ 记号 (Theta) 定义: $f(n) = \Theta(g(n))$ 表示同时有 $f(n) = O(g(n))$ 和 $f(n) = \Omega(g(n))$ 。

含义: $f(n)$ 与 $g(n)$ 同阶增长, 描述精确增长率。

小 o 记号 (Little-o) 定义: $f(n) = o(g(n))$ 表示对任意正常数 c , 存在 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \leq c|g(n)|$ 。

含义: $f(n)$ 的增长严格慢于 $g(n)$ 。

例子: $2n = o(n^2)$ 。

ω 记号 (Little-omega) 定义: $f(n) = \omega(g(n))$ 表示对任意正常数 c , 存在 n_0 , 使得对所有 $n \geq n_0$ 有 $|f(n)| \geq c|g(n)|$ 。

含义: $f(n)$ 的增长严格快于 $g(n)$ 。

常见增长率排序 (从慢到快) 常数 $O(1) <$ 对数 $O(\log n) <$ 线性 $O(n) <$ 线性对数 $O(n \log n) <$ 多项式 $O(n^c)$ ($c > 1$) $<$ 指数 $O(c^n)$ ($c > 1$) $<$ 阶乘 $O(n!)$ 等。

注意: 比较增长率时需谨慎, 如 $2^{\log^2 n}$ 虽为指数形式, 但因指数是 $\log^2 n$, 故实际增长比多项式快但比真指数 (如 e^n) 慢。

1.2 字母表和语言 (Alphabets and Languages)

基本概念

- 字母表 Σ : 有限符号的集合, 如 $\Sigma = \{0, 1\}$ 。
- 字符串: 字母表中符号的无穷序列, 空串记为 ε 。
- 克林闭包 Σ^* : 由 Σ 中符号构成的所有字符串 (含空串) 的集合。
- 语言: Σ^* 的子集, 即一些字符串的集合。

语言运算

- 并、交、补: 与集合运算相同。
- 连接: $L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\}$ 。
- 克林星号: $L^* = \{w_1 w_2 \dots w_k \mid k \geq 0, w_i \in L\}$, 即 L 中字符串的任意有限次连接。 $L^+ = L L^*$ 表示至少一次连接。(这是因为克林星号允许空串, 这里的 L^+ 正闭包强制一个因子来自 L , 不允许空串存在)

1.3 问题编码 (Encoding of Problems)

在计算理论中, 将问题实例编码为字符串以便计算模型 (如图灵机) 处理。编码应合理、简洁, 且能在多项式时间内解码。

二、作业题目详细讲解

题目 1

题目

证明

$$\frac{1}{2 \ln 2} + \frac{1}{3 \ln 3} + \dots + \frac{1}{n \ln n} = \ln \ln n + O(1)$$

和

$$\frac{1}{1^{1.1}} + \frac{1}{2^{1.1}} + \dots + \frac{1}{n^{1.1}} = O(1).$$

讲解

第一个等式证明 估计级数 $\sum_{k=2}^n \frac{1}{k \ln k}$ 。考虑函数 $f(x) = \frac{1}{x \ln x}$ ，在 $[2, \infty)$ 上为正且单调递减。对单调递减正函数，有

$$f(k+1) \leq \int_k^{k+1} f(x) dx \leq f(k) \quad (k \geq 2).$$

从 $k=2$ 累加到 n ，并将 $f(2)$ 单独留在上界中，得到

$$\int_2^{n+1} f(x) dx \leq \sum_{k=2}^n f(k) \leq f(2) + \int_2^n f(x) dx.$$

两侧都与 $\int_2^n f(x) dx$ 相比仅差有限常数，因此

$$\sum_{k=2}^n f(k) = \int_2^n f(x) dx + O(1).$$

计算积分：

$$\int \frac{1}{x \ln x} dx = \int \frac{1}{\ln x} d(\ln x) = \ln |\ln x| + C.$$

所以

$$\int_2^n \frac{1}{x \ln x} dx = \ln \ln n - \ln \ln 2.$$

因此

$$\sum_{k=2}^n \frac{1}{k \ln k} = \ln \ln n - \ln \ln 2 + O(1) = \ln \ln n + O(1),$$

其中常数 $-\ln \ln 2$ 被吸收到 $O(1)$ 中。

第二个等式证明 考虑级数 $\sum_{k=1}^n \frac{1}{k^{1.1}}$ 。由于指数 $1.1 > 1$ ，这是 p -级数且 $p > 1$ ，故无穷级数 $\sum_{k=1}^{\infty} \frac{1}{k^{1.1}}$ 收敛。收敛意味着部分和有上界，即存在常数 M 使得对所有 n 有 $\sum_{k=1}^n \frac{1}{k^{1.1}} \leq M$ ，所以 $\sum_{k=1}^n \frac{1}{k^{1.1}} = O(1)$ 。

更具体地，也可用积分估计上界：

$$\sum_{k=1}^n \frac{1}{k^{1.1}} \leq 1 + \int_1^n \frac{1}{x^{1.1}} dx = 1 + \left[\frac{x^{-0.1}}{-0.1} \right]_1^n = 1 + 10(1 - n^{-0.1}) \leq 11.$$

故确有界。

总结：第一个级数发散，发散速度为 $\ln \ln n$ ；第二个级数收敛，部分和有界。

题目 2

题目

将以下函数按渐近增长率排序（若 $f(n) = O(g(n))$ ，则 f 排在 g 前面）： $n^n - \sqrt{n} - \log \log n - 0.001n^7 - 5000n - e^n - 2^{\log^2 n} - 10^{10^{10}} - n! - 2^{2^n}$

讲解

需比较函数增长率，按从慢到快排序。分析各函数：

1. $10^{10^{10}}$ ：常数，增长率最低。
2. $\log \log n$ ：双对数，增长极慢，但比常数快。
3. $\sqrt{n} = n^{0.5}$ ：多项式（指数 0.5），比对数函数快（任何 n^c ($c > 0$) 均比 $\log n$ 快，而 $\log \log n$ 更慢）。
4. $5000n$ ：线性函数 n^1 ，比 $n^{0.5}$ 快。
5. $0.001n^7$ ：7 次多项式，比线性快。
6. $2^{\log^2 n}$ ：注意 $f(n) = 2^{\log^2 n}$ ，取对数得 $\log f(n) = \log^2 n$ 。多项式 n^c 的对数为 $c \log n$ ， $\log^2 n$ 增长比 $c \log n$ 快，故 $2^{\log^2 n}$ 比任何多项式快。但与指数函数 e^n 比较： $\log(e^n) = n$ ， n 增长比 $\log^2 n$ 快得多，所以 $2^{\log^2 n}$ 比 e^n 慢。因此它介于多项式与指数之间。
7. e^n ：指数函数，增长快。
8. $n!$ ：阶乘。由斯特林公式 $n! \sim \sqrt{2\pi n}(n/e)^n$ ，取对数 $\log(n!) \sim n \log n - n$ ，而 $\log(e^n) = n$ ， $n \log n$ 增长快于 n ，故 $n!$ 比 e^n 快。
9. n^n ：增长很快。
10. 2^{2^n} ：双指数函数，取对数得 $\log(2^{2^n}) = 2^n$ ，而 $\log(n^n) = n \log n$ ， 2^n 远快于 $n \log n$ ，故 2^{2^n} 比 n^n 快。

排序（从慢到快）：

$$10^{10^{10}} \prec \log \log n \prec \sqrt{n} \prec 5000n \prec 0.001n^7 \prec 2^{\log^2 n} \prec e^n \prec n! \prec n^n \prec 2^{2^n}$$

其中 $\prec$ 表示增长率小于。

关键点验证： $2^{\log^2 n}$ ：比多项式快（因为 $\log^2 n$ 比 $c \log n$ 快），但比指数慢（因为 $\log^2 n$ 比 n 慢）。 $n!$ 与 e^n ： $\log(n!) \sim n \log n > n = \log(e^n)$ ，故 $n!$ 更快。 n^n 与 2^{2^n} ： $\log(n^n) = n \log n$ ， $\log(2^{2^n}) = 2^n$ ， 2^n 更快。

题目 3

题目

令 $\Sigma = \{0, 1\}$ ，定义语言

$$L = \{w \in \{0,1\}^* : w \text{ 中 } 0 \text{ 和 } 1 \text{ 的数量不相等}\}.$$

证明 $L^* = \Sigma^*$ 。

讲解

需证集合相等： $L^* \subseteq \Sigma^*$ 且 $\Sigma^* \subseteq L^*$ 。

1. $L^* \subseteq \Sigma^*$ ：显然，因 L 中字符串均由 0 和 1 组成，故 L^* 中字符串也由 0 和 1 组成，属于 Σ^* 。
2. $\Sigma^* \subseteq L^*$ ：需证任意 $w \in \Sigma^*$ 可表示为 L 中若干字符串的连接。分情况：
 - 若 w 中 0 和 1 数量不相等，则 $w \in L \subseteq L^*$ 。
 - 若 w 中 0 和 1 数量相等，取最短前缀 x 使 x 中 0 和 1 数量差为 1，则剩余部分 y 的差为 -1 ，故 $x, y \in L$ ，从而 $w = xy \in L^*$ 。

综上， $\Sigma^* \subseteq L^*$ ，故 $L^* = \Sigma^*$ 。

复习小结：第一节课重点掌握渐近记号的含义与比较、积分估计级数和方法、语言运算及克林闭包的概念，并能进行简单的集合相等证明。

第二节课复习资料

二、有限自动机 (Finite Automata)

2.1 有限自动机的定义

有限自动机 (FA) 是一种最简单的计算模型，它读取输入字符串，根据当前状态和输入符号决定下一个状态，最终根据结束状态是否在接受状态集合中来决定是否接受该字符串。

形式化定义 (五元组) 一个有限自动机 M 是一个五元组 $(Q, \Sigma, \delta, q_0, F)$ ：

1. Q ：有限状态集合。
2. Σ ：输入字母表 (有限符号集合)。
3. $\delta : Q \times \Sigma \rightarrow Q$ ：转移函数。给定当前状态和输入符号，输出下一个状态。
4. $q_0 \in Q$ ：初始状态。
5. $F \subseteq Q$ ：接受状态集合。

FA 如何接受字符串 设 $w = w_1 w_2 \cdots w_n$ ，其中每个 $w_i \in \Sigma$ 。 M 从 q_0 开始，依次读入每个符号，根据 δ 转移状态。读完所有符号后，若最终状态属于 F ，则 M 接受 w ；否则拒绝。

形式化： M 接受 w 当存在状态序列 $r_0, r_1, \dots, r_n$ 满足：

- $r_0 = q_0$
- $\delta(r_i, w_{i+1}) = r_{i+1}$ (对 $i = 0, \dots, n-1$)
- $r_n \in F$

正则语言 如果一个语言 L 被某个有限自动机接受, 则称 L 是正则语言。记 $L(M)$ 为 M 接受的所有字符串的集合, 即 M 识别的语言。

2.2 设计有限自动机的思路

设计 FA 的关键在于确定状态的含义。通常, 状态用来记录读入部分字符串后所处的“情形”。例如:

- 记录当前读入的二进制数模某个数的余数。
- 记录是否已经出现了某个禁止子串。
- 记录最近读入的几个字符, 以判断是否满足模式。

2.3 正则语言的封闭性

正则语言在多种运算下是封闭的, 即对正则语言进行这些运算得到的语言仍然是正则的。封闭性证明通常通过构造新的 FA 来实现。

以下是常见的封闭运算 (设 A, B 是正则语言):

1. 补运算: $\overline{A} = \{w \in \Sigma^* \mid w \notin A\}$ 。
 - 证明: 设 M 识别 A , 则交换 M 的接受与非接受状态得到 M' , M' 识别 $\overline{A}$ 。
2. 并运算: $A \cup B$ 。
 - 证明: 使用乘积自动机。设 M_1 识别 A , M_2 识别 B , 构造 M 的状态为 M_1 和 M_2 状态的笛卡尔积 (也就是一个状态变为“二元”的), 转移模拟两者并行运行, 接受状态定义为只要有一个自动机接受即可。
3. 交运算: $A \cap B$ 。
 - 证明: 类似并运算, 但接受状态定义为两个自动机同时接受。
4. 连接运算: $A \circ B = \{xy \mid x \in A, y \in B\}$ 。
 - 证明思路: 非确定性有限自动机 (NFA) 更容易构造, 参见第三讲知识点。
5. 星号运算: $A^* = \{x_1 x_2 \cdots x_k \mid k \geq 0, \text{每个 } x_i \in A\}$ 。
 - 证明思路: 非确定性有限自动机 (NFA) 更容易构造, 参见第三讲知识点。

注意: 第二课中先介绍的是确定性有限自动机 (DFA), 但封闭性证明有时需要非确定性有限自动机 (NFA) 的概念, 可见第三课。

二、作业题目详细讲解

题目 1

题目

设计一个有限自动机, 接受以下语言

$$L = \left\{ w \in \{0, 1\}^* \mid w = w_1 w_2 \cdots w_n \text{ 使得 } \sum_{i=1}^n w_i 2^{n-i} \equiv 0 \pmod{5} \right\}.$$

换句话说，语言 L 是所有表示 5 的倍数的二进制字符串的集合，字符串以最高有效位优先（MSB）书写。绘制该有限自动机的状态图。

讲解

这个语言 L 中的字符串，如果解释为一个二进制数（最高位在前），这个数能被 5 整除。我们需要设计一个 DFA 来识别这样的字符串。

设计思路：

DFA 的状态应该记录当前已经读入的部分二进制数模 5 的余数。因为每读入一位，新的数值等于原来的数值左移一位（乘以 2）再加上新读入的位（0 或 1），然后模 5。所以，如果我们用状态 q_i 表示当前余数为 i ($i = 0, 1, 2, 3, 4$)，那么转移函数可以这样定义：

从状态 q_i 读入符号 b (0 或 1)，转移到状态 q_j ，其中 $j = (2 \times i + b) \bmod 5$ 。

初始状态应该是 q_0 ，因为还没有读入任何数字时，数值为 0，余数为 0。

接受状态也是 q_0 ，因为整个字符串读完时，我们希望余数为 0。

具体转移计算：

我们列出所有可能的状态和输入组合：

- 从 q_0 读入 0：新余数 $= (2 \times 0 + 0) \bmod 5 = 0$ ，所以转移到 q_0 。
- 从 q_0 读入 1：新余数 $= (2 \times 0 + 1) \bmod 5 = 1$ ，转移到 q_1 。
- 从 q_1 读入 0：新余数 $= (2 \times 1 + 0) \bmod 5 = 2$ ，转移到 q_2 。
- 从 q_1 读入 1：新余数 $= (2 \times 1 + 1) \bmod 5 = 3$ ，转移到 q_3 。
- 从 q_2 读入 0：新余数 $= (2 \times 2 + 0) \bmod 5 = 4$ ，转移到 q_4 。
- 从 q_2 读入 1：新余数 $= (2 \times 2 + 1) \bmod 5 = 5 \equiv 0$ ，转移到 q_0 。
- 从 q_3 读入 0：新余数 $= (2 \times 3 + 0) \bmod 5 = 6 \equiv 1$ ，转移到 q_1 。
- 从 q_3 读入 1：新余数 $= (2 \times 3 + 1) \bmod 5 = 7 \equiv 2$ ，转移到 q_2 。
- 从 q_4 读入 0：新余数 $= (2 \times 4 + 0) \bmod 5 = 8 \equiv 3$ ，转移到 q_3 。
- 从 q_4 读入 1：新余数 $= (2 \times 4 + 1) \bmod 5 = 9 \equiv 4$ ，转移到 q_4 。

状态图描述：

- 状态集合： $\{q_0, q_1, q_2, q_3, q_4\}$
- 输入字母表： $\{0, 1\}$
- 初始状态： q_0
- 接受状态： q_0
- 转移函数如上表。

我们可以绘制状态图：五个状态围成一个圆圈（顺序可为 q_0, q_1, q_2, q_3, q_4 ），每个状态根据上述转移有两条出边。注意 q_0 是初始状态（通常用箭头指向）和接受状态（通常用双圈表示）。

重要提醒：这个 DFA 也会接受空串 ε ，因为初始状态就是接受状态。在二进制数的表示中，空串通常不被视为合法的数字表示，但题目没有特别说明，按照题目定义，空串对应的数值为 0， $0 \bmod 5 = 0$ ，所以应该接受。如果要求非空字符串，则需要稍作修改，但题目没有此要求，所以这样设计是合理的。

题目 2

题目

设计一个有限自动机，识别以下语言

$$L = \{w \in \{0, 1\}^* : w \text{ 不包含子串 } 10\}.$$

绘制其状态图。

讲解

语言 L 包含所有不出现子串“10”的二进制字符串。也就是说，字符串中一旦出现了 1，后面就不能再出现 0，否则就会形成“10”。因此，合法的字符串必须是这样的形式：先有零个或多个 0，然后有零个或多个 1，即所有 0 都在所有 1 的前面。用正则表达式表示就是 0^*1^* 。

设计思路：

我们可以用三个状态来设计 DFA：

- 状态 q_0 ：表示目前还处于“0 的阶段”，即到目前为止读入的部分全部是 0（或者还没有读入任何字符）。在这个状态，我们还没有遇到 1。
- 状态 q_1 ：表示我们已经读到了至少一个 1，并且还没有出现 0 跟在 1 后面的情况。在这个状态，我们只希望后面继续读入 1，如果读到 0，就违反规则。
- 状态 q_2 ：表示已经出现了禁止子串“10”，即进入了“死亡状态”，一旦进入这个状态，无论读入什么符号，都将永远停留在这个状态，并且这个状态不是接受状态。

初始状态是 q_0 ，因为开始时还没有读入任何字符，属于 0 的阶段。

接受状态应该是 q_0 和 q_1 ，因为在这两个状态下，都没有出现禁止子串。

转移函数：

- 从 q_0 读入 0：仍然处于 0 的阶段，保持 q_0 。
- 从 q_0 读入 1：进入了 1 的阶段，转移到 q_1 。
- 从 q_1 读入 1：仍然在 1 的阶段，保持 q_1 。
- 从 q_1 读入 0：形成了“10”，禁止，转移到死亡状态 q_2 。
- 从 q_2 读入任何符号（0 或 1）：已经违规，永远留在 q_2 。

状态图描述：

- 状态集合： $\{q_0, q_1, q_2\}$
- 输入字母表： $\{0, 1\}$
- 初始状态： q_0
- 接受状态： $\{q_0, q_1\}$
- 转移函数：
 - $q_0 \xrightarrow{0} q_0$
 - $q_0 \xrightarrow{1} q_1$
 - $q_1 \xrightarrow{0} q_2$
 - $q_1 \xrightarrow{1} q_1$

$$\begin{aligned} & - q_2 \xrightarrow{0} q_2 \\ & - q_2 \xrightarrow{1} q_2 \end{aligned}$$

注意: q_2 不是接受状态, 且所有从 q_2 出发的转移都指向自己。

注意: 这个 DFA 也接受空串, 因为初始状态 q_0 是接受状态。语言 0^*1^* 确实包含空串。

题目 3

题目

使用大 O 符号的定义证明以下内容:

- 1) 如果 $f_1 = O(g_1)$ 且 $f_2 = O(g_2)$, 则 $f_1 f_2 = O(g_1 g_2)$ 。
- 2) 如果 $f_1 = O(g)$ 且 $f_2 = O(g)$, 则 $f_1 + f_2 = O(g)$ 。

讲解

我们需要回顾大 O 符号的严格定义:

$f(n) = O(g(n))$ 意味着存在正常数 C 和 N , 使得对于所有 $n > N$, 都有 $|f(n)| \leq C|g(n)|$ 。

证明第 1 小题 已知: $f_1 = O(g_1)$, 即存在常数 $C_1 > 0$ 和 N_1 , 使得对所有 $n > N_1$, 有 $|f_1(n)| \leq C_1|g_1(n)|$ 。

已知: $f_2 = O(g_2)$, 即存在常数 $C_2 > 0$ 和 N_2 , 使得对所有 $n > N_2$, 有 $|f_2(n)| \leq C_2|g_2(n)|$ 。

我们取 $N = \max(N_1, N_2)$, 那么当 $n > N$ 时, 两个已知不等式都成立。

将两个不等式相乘:

$$|f_1(n)| \cdot |f_2(n)| \leq C_1|g_1(n)| \cdot C_2|g_2(n)| = (C_1 C_2)|g_1(n)g_2(n)|。$$

由于 $|f_1(n)f_2(n)| = |f_1(n)| \cdot |f_2(n)|$, 故有

$$|f_1(n)f_2(n)| \leq (C_1 C_2)|g_1(n)g_2(n)|。$$

因此, 取 $C = C_1 C_2$, 则对于所有 $n > N$, 不等式成立。所以 $f_1 f_2 = O(g_1 g_2)$ 。

证明第 2 小题 已知: $f_1 = O(g)$, 即存在常数 $C_1 > 0$ 和 N_1 , 使得对所有 $n > N_1$, 有 $|f_1(n)| \leq C_1|g(n)|$ 。

已知: $f_2 = O(g)$, 即存在常数 $C_2 > 0$ 和 N_2 , 使得对所有 $n > N_2$, 有 $|f_2(n)| \leq C_2|g(n)|$ 。

目标: 证明 $f_1 + f_2 = O(g)$, 即要找到常数 C 和 N , 使得对所有 $n > N$, 有 $|f_1(n) + f_2(n)| \leq C|g(n)|$ 。

我们取 $N = \max(N_1, N_2)$, 那么当 $n > N$ 时, 两个已知不等式都成立。

利用三角不等式:

$$|f_1(n) + f_2(n)| \leq |f_1(n)| + |f_2(n)| \leq C_1|g(n)| + C_2|g(n)| = (C_1 + C_2)|g(n)|。$$

因此, 取 $C = C_1 + C_2$, 则对于所有 $n > N$, 有 $|f_1(n) + f_2(n)| \leq C|g(n)|$ 。所以 $f_1 + f_2 = O(g)$ 。

复习小结: 第二节课重点掌握有限自动机的形式化定义、设计方法 (特别是基于余数或禁止模式的状态设计), 以及正则语言封闭性的基本证明思路 (补、并)。同时, 大 O 记号的性质证明需严格按照定义进行构造。

第三节课复习资料

三、非确定性有限自动机（NFA）与正则表达式

3.1 非确定性有限自动机（NFA）

NFA 的定义 NFA 与 DFA 类似，但转移函数可以是非确定性的：即从一个状态读入一个符号，可以转移到多个可能的状态（包括零个）。此外，NFA 允许 ε 转移，即不读入任何符号就改变状态。

形式化定义：一个 NFA 是一个五元组 $(Q, \Sigma, \delta, q_0, F)$ ，其中：

- Q ：有限状态集合。
- Σ ：输入字母表。
- $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$ ，其中 $\mathcal{P}(Q)$ 是 Q 的幂集（即 Q 中所有状态都可能存在， 2^Q 种）。
- $q_0 \in Q$ ：初始状态。
- $F \subseteq Q$ ：接受状态集合。

NFA 接受一个字符串 w ，如果存在一条从初始状态到某个接受状态的路径，路径上的转移标记依次与 w 的符号匹配（允许 ε 转移，不消耗输入符号）。

NFA 与 DFA 的等价性 对于任意 NFA，存在一个 DFA 接受相同的语言（反之亦然）。DFA 本身显然就是一种 NFA，而反过来证明可使用子集构造法：将 NFA 的状态集合的子集作为 DFA 的状态（关键！我们构造的 DFA，其状态应是已知 NFA 的某些状态的集合），以此模拟 NFA 所有可能的并行状态。

子集构造法：给定 NFA $N = (Q, \Sigma, \delta, q_0, F)$ ，构造 DFA $M = (Q', \Sigma, \delta', q'_0, F')$ ：

- $Q' = \mathcal{P}(Q)$ 。
- $q'_0 = E(\{q_0\})$ ，其中 $E(R)$ 表示从集合 R 中的状态通过 ε 转移所能到达的状态集合（ ε 闭包），相当于把 NFA 中可以用来出发的状态“打包”了。
- 对于 $S \subseteq Q$ 和 $a \in \Sigma$ ，定义 $\delta'(S, a) = E(\bigcup_{r \in S} \delta(r, a))$ 。这样，我们就能得到我们构造的 DFA 中的一个“打包”遇到一个输入字母时，应去往哪一个“打包”。
- $F' = \{S \subseteq Q : S \cap F \neq \emptyset\}$ 。因为我们的最终目标是让构造的 DFA 和原始的 NFA 等价，因此凡是包含 F 中状态的的状态集合都应作为我们构造的 DFA 的接收状态。

正则语言的封闭性（连接与星号） 利用 NFA 可以方便地证明正则语言在连接和星号运算下封闭。

- 连接：若 L_1 和 L_2 是正则语言，则 $L_1 \circ L_2 = \{xy : x \in L_1, y \in L_2\}$ 也是正则的。证明：设 N_1 接受 L_1 ， N_2 接受 L_2 。构造 NFA N ：将 N_1 的每个接受状态通过 ε 转移连接到 N_2 的初始状态。 N 的初始状态为 N_1 的初始状态，接受状态为 N_2 的接受状态。
- 星号：若 L 是正则语言，则 $L^* = \{x_1 x_2 \cdots x_k : k \geq 0, x_i \in L\}$ 也是正则的。证明：设 N_1 接受 L 。构造 NFA N ：新增一个初始状态 q_0 （也是接受状态），从 q_0 通过 ε 转移到 N_1 的初始状态；同时从 N_1 的接受状态通过 ε 转移回到 N_1 的初始状态（这样就允许了重复）。

3.2 正则表达式

正则表达式是一种描述正则语言的形式化方法。递归定义如下：

1. 基础:

- a ($a \in \Sigma$) 是正则表达式, 表示语言 $\{a\}$ 。
- ε 是正则表达式, 表示语言 $\{\varepsilon\}$ 。(注意, 这里的 $\{\varepsilon\}$ 中只包含一个元素, 即空串, 不能和空集混淆)
- $\emptyset$ 是正则表达式, 表示空语言。

2. 归纳:

- 如果 r 和 s 是正则表达式, 表示语言 $L(r)$ 和 $L(s)$, 则:
 - $(r \cup s)$ 是正则表达式, 表示 $L(r) \cup L(s)$ 。
 - $(r \circ s)$ 是正则表达式, 表示 $L(r) \circ L(s)$ 。
 - (r^*) 是正则表达式, 表示 $L(r)^*$ 。

通常, 运算符优先级为: 星号 > 连接 > 并集。括号可省略。

正则表达式与有限自动机等价: 每个正则表达式对应一个正则语言, 反之亦然。证明可通过将正则表达式转换为 NFA (递归构造), 或反之 (状态消去法)。

三、作业题目详细讲解

题目 1

题目

定义一个反向读取 DFA 为从右向左处理输入字符串的确定性有限自动机。证明一个语言是正则的当且仅当它可以被某个反向读取 DFA 接受。

讲解

我们需要证明两个方向。

方向 1: 如果语言 L 是正则的, 那么存在一个反向读取 DFA 接受 L 。

证明: 因为 L 是正则语言, 所以存在一个普通的 DFA $M = (Q, \Sigma, \delta, q_0, F)$ 接受 L 。我们构造一个反向读取 DFA M' 如下:

- M' 的状态集与 M 相同: $Q' = Q$ 。
- M' 的输入字母表与 M 相同: $\Sigma' = \Sigma$ 。
- M' 的转移函数 δ' 定义为: 对于任意 $q \in Q$ 和 $a \in \Sigma$, 令 $\delta'(q, a) = p$ 当且仅当在 M 中有 $\delta(p, a) = q$ 。也就是说, M' 的转移是 M 转移的反向。
- M' 的初始状态设为 M 的任意一个接受状态。

上述做法在转移函数构造和初始状态构造上, 是否存在不严谨的地方呢? 实际是有的: 1) 把 $\delta'(q, a)$ 定义为“所有前驱”后, 它可能不是单值, 所得自动机其实是 NFA 而非 DFA; 2) 初始状态不能随便选取一个接受状态, 而应该同时考虑所有接受状态的反向可达性, 否则会漏掉路径。

可以考虑: 先构造一个 NFA N , 使得 N 接受 L 的反转语言 $L^R = \{w^R : w \in L\}$, 其中 w^R 是 w 的翻转。具体构造:

- 先在 M 的基础上新增一个初始状态 s ，从 s 通过 ε 转移到每个 $f \in F$ ；设 $N = (Q \cup \{s\}, \Sigma, \delta_N, s, \{q_0\})$ 。将这个 s 作为初始状态，就巧妙地解决了上述的“所有接受状态的反向可达性”问题，因为 s 可以到达这些状态，现在就等价于 M 的最终状态了。
- 定义反向转移：对任意 $q \in Q, a \in \Sigma$ ，令 $\delta_N(q, a) = \{p \in Q : \delta(p, a) = q\}$ （这样就解决了上述“可能不是单值”问题）；其他未定义的转移为空集。
- 最后， $\{q_0\}$ 可构造为唯一接受状态。

这个 NFA N 接受的语言正好是 L^R 。因为 L 是正则的，所以 L^R 也是正则的（正则语言在反转下封闭，已证结论）。然后，将 NFA N 通过子集构造法转化为 DFA D （前面已证），这个 DFA D 接受 L^R 。最后，说明让其反向读取即可。因此，存在反向读取 DFA 接受 L 。

方向 2：如果存在一个反向读取 DFA 接受语言 L ，那么 L 是正则的。

证明：设 M' 是一个反向读取 DFA，接受 L 。定义 M' 正向读（从左向右）时接受的语言为 L' 。由于 M' 是 DFA，所以 L' 是正则语言。但注意 M' 反向读时接受 L ，正向读时实际上接受的是 L 的反转 L^R 。因此 $L^R = L'$ 是正则的。由于正则语言在反转下封闭，所以 $L = (L^R)^R$ 也是正则的。因此直接得证。

综上，语言 L 是正则的当且仅当它可以被某个反向读取 DFA 接受。

题目 2

题目

给出语言 $L = \{w \in \{0,1\}^* : w \text{ 中 } 1 \text{ 的个数是偶数, 且 } 0 \text{ 的个数也是偶数}\}$ 的正则表达式。

讲解

我们想要所有包含偶数个 0 和偶数个 1 的二进制字符串。考虑用模 2 的状态来思考。但直接写正则表达式，可以注意到：任何这样的字符串，都可以看成由以下四种“片段”组成，这些片段在添加时不会改变 0 和 1 的奇偶性（即添加后，0 和 1 的个数的奇偶性保持不变）：

- “00”：添加两个 0，不改变奇偶性。
- “11”：添加两个 1，不改变奇偶性。
- “01”后跟“10”：即“0110”，整体添加了两个 0 和两个 1，奇偶性不变。类似地，“10”后跟“01”也是。

因此，最简洁的表达式是：

$$(00 \cup 11 \cup (01 \cup 10)(00 \cup 11)^*(01 \cup 10))^*$$

解释：

- $00 \cup 11$ ：添加两个相同字符，奇偶性不变。
- $(01 \cup 10)(00 \cup 11)^*(01 \cup 10)$ ：先添加一个“01”或“10”，这会翻转 0 和 1 的奇偶性（使 0 和 1 都变成奇数个），然后添加任意个“00”或“11”（不改变奇偶性），最后再添加一个“01”或“10”，再次翻转奇偶性，回到偶数。整体效果是奇偶性不变。

整个表达式外面加星号，表示重复任意多次（包括零次），所以最终得到的就是所有偶数个 0 和偶数个 1 的字符串。

注意：这个表达式也包含空串，因为星号允许零次重复，而空串中 0 和 1 的个数都是 0（偶数），符合要求。

题目 3

题目

构造与以下正则表达式对应的 NFA: 1. $0^+ \cup (01)^*$ 2. $(0 \cup 1^+)0^*1^+$

讲解

我们分别构造两个正则表达式的 NFA。NFA 允许 ϵ 转移，并且每个部分可以独立构造然后组合。

第一问: $0^+ \cup (01)^*$ 这个语言表示: 要么是一个或多个 0 的序列, 要么是零个或多个 “01” 的序列。

我们构造 NFA 如下:

- 状态集合: $\{q_0, q_1, q_2, q_3, q_4, q_5\}$, 其中 q_0 是初始状态。
- 输入字母表: $\{0, 1\}$ 。
- 接受状态: q_2 (对于 0^+ 部分) 以及 q_3 和 q_5 (对于 $(01)^*$ 部分)。

转移函数:

1. 从 q_0 通过 ϵ 转移到 q_1 (进入 0^+ 分支) 和通过 ϵ 转移到 q_3 (进入 $(01)^*$ 分支)。
2. 0^+ 部分:
 - $q_1 \xrightarrow{0} q_2$ 。
 - $q_2 \xrightarrow{0} q_2$ (自环, 允许更多 0)。
3. $(01)^*$ 部分:
 - q_3 是接受状态 (因为 $(01)^*$ 允许空串)。
 - $q_3 \xrightarrow{0} q_4$ 。
 - $q_4 \xrightarrow{1} q_5$ 。
 - q_5 是接受状态。
 - $q_5 \xrightarrow{0} q_4$ (以便继续下一个 “01”)。

这样, 对于 $(01)^*$, 空串可以直接在 q_3 接受; 非空的 “01” 序列会经过 $q_3 \rightarrow q_4 \rightarrow q_5$, 并且从 q_5 读 0 可以回到 q_4 继续循环。

第二问: $(0 \cup 1^+)0^*1^+$ 这个语言表示: 首先是一个 0 或者一个或多个 1, 然后是零个或多个 0, 最后是一个或多个 1。

我们构造 NFA 如下:

- 状态集合: $\{q_1, q_2, q_3, q_4, q_5\}$, 其中 q_1 是初始状态。
- 输入字母表: $\{0, 1\}$ 。
- 接受状态: q_5 。

转移函数:

1. 第一部分 $(0 \cup 1^+)$:
 - $q_1 \xrightarrow{0} q_2$ (选择 0)。
 - $q_1 \xrightarrow{1} q_3$ (选择 1^+)。
 - $q_3 \xrightarrow{1} q_3$ (自环, 允许更多 1)。

2. 连接第二部分 0^* :

- 从 q_2 和 q_3 通过 ε 转移到 q_4 。
- $q_4 \xrightarrow{0} q_4$ (自环, 允许任意多个 0)。

3. 连接第三部分 1^+ :

- $q_4 \xrightarrow{1} q_5$ (第一个 1)。
- $q_5 \xrightarrow{1} q_5$ (自环, 允许更多 1)。

这样, NFA 从 q_1 开始, 根据第一个字符选择分支, 然后通过 ε 转移到 q_4 , 在 q_4 可以读任意多个 0, 最后必须读至少一个 1 到达 q_5 并可以继续读更多 1。 q_5 是接受状态。

注意: 这个构造中, 从 q_2 和 q_3 到 q_4 的 ε 转移是必要的, 因为 0^* 部分允许零个 0, 所以不能直接合并状态。而 q_4 到 q_5 的转移是读 1, 没有使用 ε 转移, 这样更简洁。

复习小结: 第三节课重点掌握 NFA 的定义及其与 DFA 的等价性 (子集构造法)、正则表达式的递归定义、以及利用 NFA 证明正则语言在连接和星号运算下的封闭性。同时, 要能够进行正则表达式与有限自动机之间的转换 (构造 NFA)。作业题目涉及反向 DFA 的等价性证明、构造特定语言的正则表达式、以及从正则表达式构造 NFA, 需要理解并熟练运用相关概念和构造技巧。